

Terrain, Strata and Mining Architecture

A single data model covering player terraforming, per-cell surface materials, layered geology, ore distribution and tunnelling — with an implementation order chosen so that later stages extend earlier ones instead of reworking them.

Prepared 7 September 2026 Prototype Tests/Terraform, Unity 6000.3.9f1

Target 64+ player authoritative server Status proposal for evaluation

01 What the system has to do

Restating the requirement precisely, because most of the architecture follows from one line of it.

- **Terraform.** Raise, lower and flatten ground at cell resolution, with results that hold their shape and read as natural.
- **Surface material.** Rock, sand, soil, forest soil, clay, slate and so on — painted at world build, then changed by players at roughly cell granularity.
- **Place and recover.** Dropping material raises the ground and changes its surface. Digging it back lowers the ground, returns the material to inventory, and *exposes whatever was underneath*.
- **Depth matters.** Stack soil, clay, then rock and the surface reads rock; dig down and clay appears, then soil. Lower layers are tracked, not just the top.
- **Geology.** Ore veins concentrated in mountains and ridges, rock throughout, clay in its own areas — mixed within a single cell volume (20% iron, 80% rock).
- **Mining.** Native rock cannot be shovelled. Removing it is a mining action that opens a void, and the void exposes the surrounding cells' interiors.
- **Authored worlds.** Initial worlds built with an external tool so several maps can be offered, with the sub-surface implied rather than hand-placed.

The line that decides the architecture is *"the material layers underneath would be logical until exposed."* Taken seriously, it means the strata are not data at all.

02 The central split: computed vs authored

The instinct to run two tracking systems is right, but the productive seam is not *surface versus deep*. It is **what a function can produce** versus **what only a player could have produced**.

	GEOLOGY	EDITS
Origin	A pure function of cell and world seed	Player actions
Extent	Every cell, every depth, to bedrock	A sparse skin over touched cells only
Mutation	Never written. Only occluded by removals	Written constantly, added and removed
Storage	Zero bytes. Evaluated on demand	Explicit spans, only where touched
Replication	Never sent. Identical on every client from the seed	Every change, as commands
Persistence	The seed and the map file	The save

Three consequences are worth stating plainly, because they are the reason to prefer this over a stored voxel or column world.

An untouched world stores nothing per cell. A cell allocates the first time somebody digs it. A busy village might touch ten thousand cells; that is on the order of a hundred kilobytes for the whole settlement, against hundreds of megabytes to store the same region as columns.

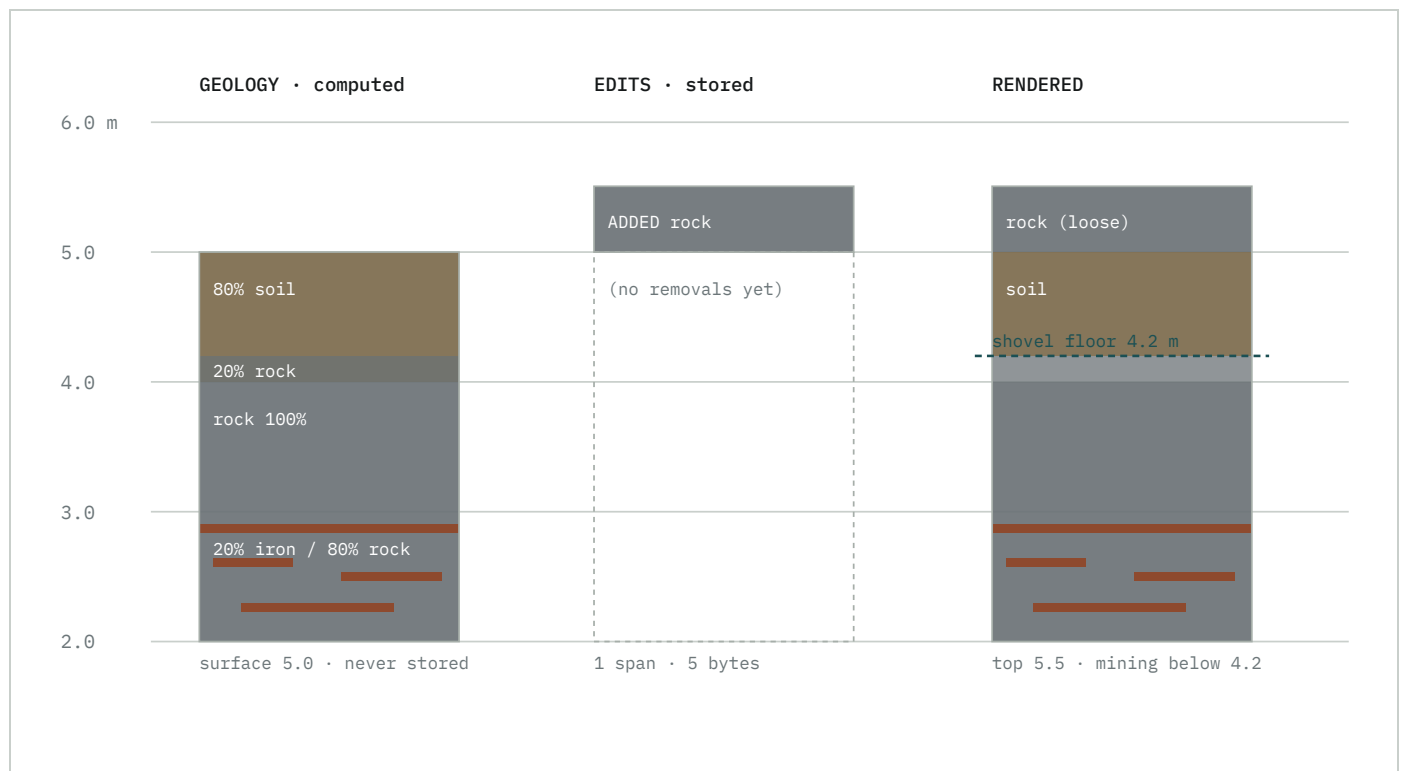
"Was this placed by a player?" needs no flag. It is answered by which structure the material lives in. The rule that native rock cannot be shovelled but mined rock can be re-dug falls out of the split rather than being enforced by a per-run boolean that has to be kept honest through every operation.

Geology never goes on the wire. Clients derive it. What replicates is the edit layer, which was going to replicate anyway.

COST OF THIS CHOICE

Geology must be *bit-identical* on server and every client, forever, across platforms and engine versions. That rules out `Mathf.PerlinNoise` and every other engine-provided noise, none of which guarantee stability across versions. The generator has to be integer hash noise the project owns and can freeze. This is cheap to do on day one and near-impossible to retrofit once worlds exist.

03 The column, in both layers



The worked example. Geology is a function evaluated on demand — the whole left column costs nothing to store. The player's pile of mined rock is the only thing written down. What renders is the two composed. Digging to 4.2 m adds a removal span; the geology is never edited.

Why the shovel floor sits at 4.2

The run from 4.0 to 5.0 is 80% soil and 20% rock. Composition is read as **sorted by hardness**: the soft fraction comes off first and the hard fraction is left behind. Eighty per cent of a one-metre run is 0.8 m, so a shovel reaches 4.2 m and stops. Below that the cell is rock and the action becomes mining.

This one rule does a surprising amount of work. It produces the "cannot dig through iron" behaviour with no special case, it gives ore-bearing rock a natural resistance proportional to its grade, and it means a mixed run yields *proportional* quantities of both materials when it is finally mined out.

04 Data structures

All vertical quantities use the existing fixed-point unit — `1/20 m`, one `ushort`, exact to a tenth of a metre. Heights, run boundaries and void extents share it. Nothing anywhere holds a vertical float.

```

// ---- geology: computed, never stored -----

struct Composition {
    byte    Primary;           // material id
    byte    Secondary;         // material id, or None
    byte    SecondaryPct;      // 0..100, hardness-sorted
}

struct StrataRun {
    ushort TopRaw;             // 1/20 m, exclusive upper bound
    ushort BottomRaw;
    Composition What;
}

// Deterministic. Same result on server and every client, forever.
StrataColumn(cx, cz) -> StrataRun[]

// ---- edits: stored, sparse -----

enum EditKind : byte { Removed, Added }

struct EditSpan {             // 5 bytes packed
    ushort TopRaw;
    ushort BottomRaw;
    byte    KindAndMaterial;    // 1 bit kind, 7 bits material
}

class CellEdits {
    EditSpan[] Spans;           // ordered, top to bottom, non-overlapping
}

// Untouched cells have NO entry. This is the whole point.
Dictionary<int, CellEdits> _edits;

```

A single ordered span list per touched cell covers every case uniformly. A dug pad is one `Removed` span at the top. A pile is one `Added` span. A tunnel is a `Removed` span with geology above it. There is no separate cut depth, no separate void list, and no third structure to keep in agreement with the other two.

CASE	SPANS	BYTES	NOTES
Untouched cell	0	0	No dictionary entry at all
Levelled build pad	1	5	One removal from the top
Pile of spoil	1	5	One addition; same material merges
Cut and filled	2	10	Removal, then addition above it
Tunnel under intact ground	1	5	Removal with geology above
Village of 10,000 touched cells	~20k	~100 KB	Whole settlement, all edits

Resolving a cell

Everything the renderer and the tools need comes from composing the two layers top-down. The common case — a cell with no edits — never allocates and never touches the dictionary.

```
// Surface height and the material that shows there.
Surface(cx, cz):
    edits = _edits.TryGet(cx, cz)          // usually absent
    if no edits:
        run = StrataColumn(cx, cz)[0]
        return (run.TopRaw, run.What.Primary)

    for span in edits.Spans:               // already top-down
        if span.Kind == Added:
            return (span.TopRaw, span.Material)

    // otherwise: geology, minus whatever has been removed from it
    return FirstIntactPoint(StrataColumn(cx, cz), edits.Spans)
```

WHY A COLUMN FUNCTION, NOT A POINT FUNCTION

The geology generator must return the *runs of a column*, not the composition at a single point. A point sampler cannot answer "how far down can a shovel reach" without integrating over depth, and that question is asked on every dig. Returning runs makes the shovel floor a subtraction.

05 The geology generator

Four layers, evaluated in order, all deterministic from the map file plus the cell coordinate. Each is independently authorable, which is what keeps world building in the external tool rather than in code.

1. **Surface height** — read from the map's heightmap. This is what the external tool authors, and it is already what the current importer samples.
2. **Surface material** — read from the map's material map, one byte per cell, classified from the tool's painted layers at build time. Sand, rock, soil, forest soil, clay, slate.
3. **Depth profile** — topsoil thickness, then subsoil, then bedrock, with thicknesses driven by the surface material and by slope. Steep ground gets thin soil because it sheds; hollows get deep soil because they collect.
4. **Veins** — hash noise thresholded against a mask built from terrain features. Ore where the mask says ridge and elevation is high; clay where it says low and flat. The "heatmap" is this mask, and it is computed rather than painted.

Layers 1 and 2 come from the map file. Layers 3 and 4 are parameters plus a seed. A map is therefore a heightmap, a byte-per-cell material map, and a small parameter block — nothing a designer has to author by hand below the surface.

The mask answers the mineral-distribution requirement directly. "Ore veins in mountains and ridges" is a ridge measure taken from the heightmap — curvature, or elevation percentile within a radius — multiplied by a depth term. It needs no separate authored data and it stays consistent with the terrain automatically, including on maps built later.

06 Operations

Every action keeps the existing shape, with one stage added. `Validate` stays pure and cheap because it runs on hover for the HUD as well as on commit.

STAGE	RESPONSIBILITY	PURITY
Validate	May this happen at all? Bounds, reach, step guard, occupancy, tool tier against material hardness	Pure, read-only, allocation-free
Resolve	What exactly happens? Which spans change, where sluffed material lands, what quantities enter the inventory	Pure, deterministic, bounded iteration
Apply	Write the resolved spans, mark the dirty region	The only writer

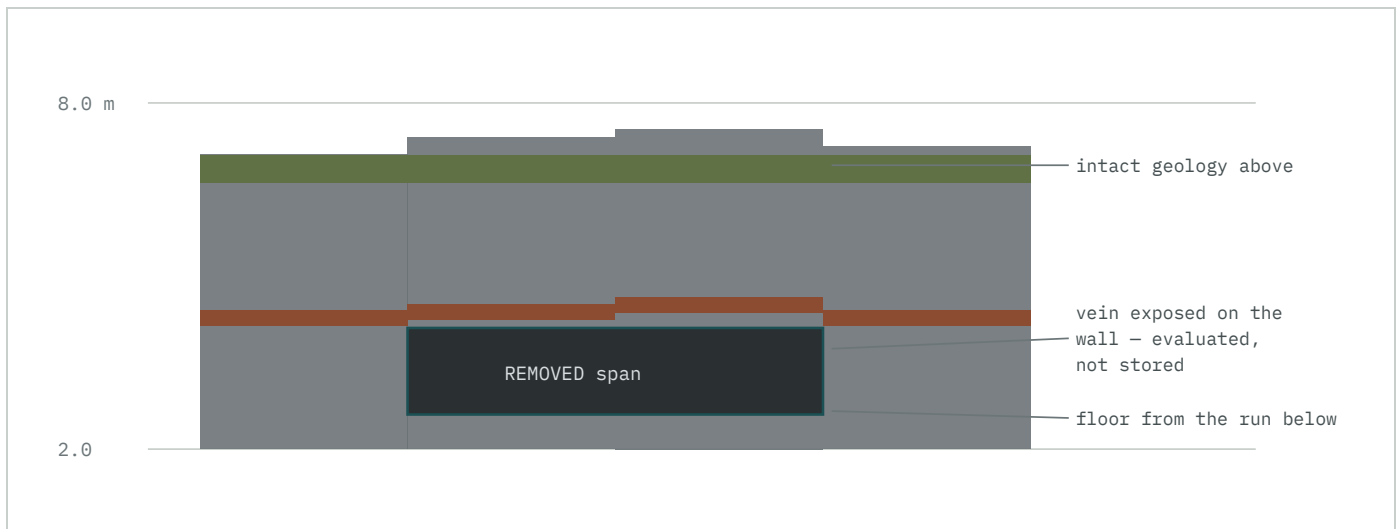
Resolve is new and it is what makes sluffing work. Today a blocked edit is refused. With a Resolve stage the edit succeeds and only its destination changes — material that cannot rest slides to the lower neighbour, which is better feel and no dead clicks. Because Resolve is pure and deterministic, the server runs it and the client can run the same code for prediction and get the same answer.

Two properties Resolve must hold, both easy to guarantee now and expensive to add later: a **fixed traversal order**, so an avalanche settles identically everywhere, and a **hard iteration cap**, so one click cannot cascade unboundedly across a map.

The three verbs

- **Place** — consume from inventory, add or extend an **Added** span. Same material as the span below merges into it, which is what keeps span counts flat under repeated piling.
- **Dig** — shovel work. Permitted through **Added** spans without restriction, and into native ground only down to the shovel floor of the current run. Yields the material removed.
- **Mine** — gated on tool tier against the run's hardest constituent. Removes a bounded volume, yields proportional quantities of every constituent, and writes a **Removed** span. When that span has intact geology above it, a void exists and the cell is no longer a heightfield.

07 Voids, and where the model changes shape



A tunnel is one removed span. Its walls, floor and ceiling are not stored — they are the geology function evaluated at the void's boundary, which is why breaking into a neighbouring cell exposes that cell's interior for free. The *data* supports this from day one; only the mesher has to catch up.

Up to voids, everything is still a heightfield with bookkeeping above and below one surface. A void with intact material over it is genuinely 2.5D, and that is the single largest jump in this plan — larger than composition, larger than the edit layer. It needs a mesher that emits floors and ceilings, collision for both, and navigation that understands overhead cover.

THE DECISION THAT MUST NOT BE DEFERRED

Even before any of that is built, **the surface API has to admit more than one surface per cell**. A signature like `GetHeight(cx, cz) -> ushort` bakes in the assumption that a cell has exactly one, and it will be called from the mesher, the tools, the HUD, the controller and the pathfinder before voids ship. Shaping it as `SurfacesAt(cx, cz)` — returning one entry today and several later — costs almost nothing now and is the difference between adding tunnels and rewriting to accommodate them.

08 Rendering

The current meshers already have the property that makes per-cell material free: **neither shares a vertex between cells**. The cell mesher fans eight triangles from a

centre with twenty-four fresh vertices; the vertex mesher emits six independent vertices per cell for flat normals. Both always append.

So every vertex belongs to exactly one cell. Write that cell's material index into UV2, put the material textures in a `Texture2DArray`, and the shader samples the right slice — crisp per-cell boundaries, no splatmap, no blending, no extra geometry.

- **Composition variants** are bands of the secondary percentage. Rock with a trace of iron, rock with visible iron, near-solid vein: three slices chosen by threshold, which is exactly the set of textures already envisaged.
- **A material change needs no geometry rebuild.** Geometry has not moved, so the UV2 buffer is rewritten and nothing else — no triangle rebuild and, critically, no collider re-cook. Painting is an order of magnitude cheaper than terraforming.
- **Void surfaces** take their material from the geology evaluated at the wall, through the same array and the same shader.

BUILD HAZARD

This is the point where a custom shader stops being avoidable, and custom shaders are how a player build renders magenta. The mitigation already exists — `Assets/Editor/BuildPrep.cs` forces shaders into Always Included Shaders before every build — but the new shader has to be added to its list, and that is easy to forget until a tester reports a pink world.

09 Replication and persistence

The split does most of the work here. Geology is derived from the seed on every machine and never transmitted. What crosses the wire is what crossed it before: commands.

PATH	CARRIES	ORDER OF SIZE
Player edit, broadcast	Command: verb, cell, quantity, material	~12 B
Chunk stream on approach	Edit spans for that chunk only	0 – 20 KB
Untouched chunk	Nothing. The seed already describes it	0 B
Returning player, absent a month	Current edit spans for chunks they load	as above
Save file, whole world	Seed, map reference, all edit spans	grows with play, not with world size

Two properties fall out that are hard to get any other way. **World size is free** — a sixteen-square-kilometre map costs the same to store as a one-kilometre map until players dig. And **a returning player sees the world as it is**, because the state they receive is the current span set, not a replay of everything that happened while they were gone.

The server does no meshing, so none of the rendering cost in section 08 applies to it. Server work per edit is a Validate, a Resolve over a bounded region, and a span write.

10 What is load-bearing, and what is not

The concern about phasing is well founded, but the risk is not evenly spread. Six decisions contaminate everything downstream if they change. The rest can be added later without touching what came before. Settling the six — on paper, now — is what makes phasing safe.

Load-bearing — settle before writing much more code

L1 Geology computed, not stored

Determines storage, replication, save format, and whether a large persistent world is affordable at all. Reversing it later means rewriting persistence and streaming.

L2 **One vertical quantum everywhere**

Heights, run boundaries and void extents all in 1/20 m integers. Mixed units produce seams where a tunnel ceiling misses a cell surface, and the fix touches every file.

L3 **Surface APIs admit more than one surface**

The quiet one. `GetHeight` returning a single value will be called from a dozen systems before tunnels exist, and every one becomes a rewrite. See section 07.

L4 **Validate / Resolve / Apply**

Sluffing, mining yields and inventory all need an outcome to be computed rather than merely approved. Adding the stage later means revisiting every operation written without it.

L5 **Owned, frozen noise**

Integer hash noise the project controls. Engine noise is not stable across versions, and a change to it silently rewrites the geology under every existing save.

L6 **Cell model or vertex model**

Everything above sits on it, and it is the one decision the existing playtest already answers. Open the survey, close it, then build.

Deferrable — add later without rework

FEATURE	WHY IT IS SAFE TO DEFER
Composition and ore grades	Widens the geology function's return type. No other system reads it structurally.
Material list and hardness values	Data. Adding slate later is a row in a table.
Texture variants and the shader	Presentation over an index the mesher already writes.
Sluffing	A Resolve-stage behaviour, provided L4 exists.
Occupancy blocking	A Validate-stage predicate. One more read.
Inventory	Consumes yields Resolve already computes.
Veins and the mineral mask	Parameters to the geology function.
Tunnel <i>rendering</i>	The data model admits voids from day one; only the mesher lags, provided L3 exists.

THE RECOMMENDATION

A full-scale prototype is not necessary, and building one would mostly re-litigate decisions that a paper answer settles more cheaply. What is necessary is that the six load-bearing decisions are made *before* stage two, and that the interfaces are shaped for the end state even where the implementation is not. That is a smaller commitment than a full build and it removes most of the rework risk that phasing would otherwise carry.

11 Implementation order

Each stage is ordered so that it can be evaluated on its own, and so that a bad result invalidates as little of the previous stage as possible.

STAGE 0

Close the model survey

Cell model or vertex model, decided by testers on the build that already exists. It is the cheapest of all these decisions and every later stage sits on it.
Settles L6 · nothing new is built

STAGE 1

Geology, read-only

The column function, the map format, the import that classifies painted layers into surface materials, and the texture array. No edits at all — walk a generated world whose strata are computed and whose surface is correctly textured.

Proves L1 L2 L5 and the authoring pipeline · risks the shader

STAGE 2

The edit layer

Spans, place and dig, the shovel floor, inventory yields. Shaped against the multi-surface API even though every cell still has exactly one surface.

Proves L3 L4 and the two-layer split · first playable loop

STAGE 3

Authority and persistence

Server-side Validate and Resolve, command replication, chunk streaming, save and reload. Still single-surface, so the hard networking problem is isolated from the hard geometry one.

Proves the replication model at scale · deliberately before tunnels

STAGE 4

Voids and mining

The 2.5D mesher, void collision, wall materials from geology, tool tiers. The largest single jump, taken against a data model that has already supported it for three stages.

The real risk · isolated on purpose

STAGE 5

Behaviour

Sluffing, occupancy blocking, angle of repose as a material property, ore grades and variant textures. All additive, all landing in stages that already exist.

Nothing structural · tuning and feel

Stage 3 before stage 4 is the deliberate call. Both are hard; doing them together means a desync and a meshing bug look the same from the outside. Networking a single-

surface world is a solved-shape problem, and tunnels then arrive as geometry over a replication layer that is already trusted.

12 Open questions

- **Is there bedrock?** An unremovable bottom run gives a natural dig limit and keeps every height positive. Without one, "what is at the bottom" needs another answer.
- **What happens when sluffed material lands on a different material?** A new span on top is the obvious rule, but a hillside taking repeated thin sluffs will grow span counts. Merging same-material arrivals covers most of it.
- **Does mined rock re-place as rock?** If loose spans are single-material, a mixed run mines into two inventory items and returns as two pure spans. That is simpler and probably right, but it means you cannot put the hillside back exactly as it was.
- **How thin can a span be?** The quantum allows 0.05 m. Very thin spans are individually cheap but pathological in aggregate; a minimum thickness with merge-on-place is probably wanted.
- **Do voids collapse?** Unsupported ceilings are a whole system. Worth deciding whether it is ever in scope, because "no" is a legitimate answer and it simplifies stage 4.
- **Structures over voids.** Occupancy is a per-cell idea today. A building over a tunnel makes it a per-surface idea, which is another consumer of L3.

Groundworks terrain systems · proposal for evaluation, 7 September 2026.

Written against the prototype at Tests/Terraform (Unity 6000.3.9f1, 64×64 cells at 1 m, heights in 1/20 m fixed point). Sizes quoted are order-of-magnitude estimates from the structures described here, not measurements.